

Domanda A (6 punti)

1. Dare la definizione della notazione Ω , cioè, date due funzioni $f(n)$ e $g(n)$, definire il significato della notazione $f(n) = \Omega(g(n))$.
2. Ordinare le seguenti funzioni per ordine di grandezza decrescente, cioè scrivere le funzioni secondo un ordine $f_1, f_2, \dots, f_8$ tale che risulti $f_1 = \Omega(f_2), f_2 = \Omega(f_3), \dots, f_7 = \Omega(f_8)$.

$n^{2/3}$ 10 $\frac{n}{\sqrt{n}}$ 1.1^n $\frac{n}{2^n}$ n^2 $\sqrt{\log n}$ $\log n$

GERARCHIA

$n^{1/2}$

① INFINITESIMO $\Rightarrow$ TONDO A ZERO $\frac{M}{2^n}$

② $K \equiv$ COSTANTE $\rightarrow$ HASH

③ LOGARITMICO / $\log K/n$

④ SUBLINEARE $\rightarrow -n^{1/2} \approx \sqrt{n}$
 $-n^{0.9} / n^2$

⑤ LINEARE $\rightarrow n^2 / n \log^2 n / n \log \dots n$

⑥ CUBICI / QUADRATICI

⑦ ESPONENZIALI $\equiv$ BRUTE FORCE (LCS)

Domanda A (6 punti)

1. Dare la definizione della notazione O , cioè, date due funzioni $f(n)$ e $g(n)$, definire il significato della notazione $f(n) = O(g(n))$.
2. Ordinare le seguenti funzioni per ordine di grandezza crescente, cioè scrivere le funzioni secondo un ordine $f_1, f_2, \dots, f_8$ tale che risulti $f_1 = O(f_2), f_2 = O(f_3), \dots, f_7 = O(f_8)$.

$n^{3/2}$ 2^n $n \log^2 n$ $\log n$ $n^{0.9}$ $(\log n)^3$ 1.05^n $\frac{n^2}{\log n}$

Esercizio 2 (9 punti) Avete n file $f_1, f_2, \dots, f_n$ di dimensioni $d_1, d_2, \dots, d_n$ MB da caricare sul vostro servizio di archiviazione cloud. Purtroppo la somma delle dimensioni di questi file è superiore alla capacità di c MB disponibile su cloud, e non volete pagare per ottenere più spazio. Vi dovete quindi accontentare di caricare il maggior numero possibile di file, senza eccedere la capacità c .

- (a) Progettare un algoritmo greedy per questo problema, e valutarne la complessità.
 (b) Enunciare e dimostrare la proprietà di scelta greedy su cui l'algoritmo si basa.

GREEDY_UPLOAD (f, d, c)

// Ordinamento crescente dei file ($F_1, \dots, F_n$)

opt = { d_1 }
 size_used = 0

while opt < c
 if size_used + $d_i \leq c$
 opt = opt + f_i
 size_used += d_i
 i++

return opt

GREEDY-FILE-UPLOAD(F, d, c)

1. Ordina i file in F crescenti
2. spazio_usato $\leftarrow 0$
3. file_caricati $\leftarrow \emptyset$
4. for $i = 1$ to n do
5. if spazio_usato + $d[i] \leq c$ then
6. file_caricati $\leftarrow$ file_caricati $\cup \{f[i]\}$
7. spazio_usato $\leftarrow$ spazio_usato + $d[i]$
8. return file_caricati

$O(n)$

$\Theta(n \log n)$!

(b) - $f_1 \dots f_n$
 - $d_1 \dots d_n$
 - $c = \text{CAPACITÀ}$

CUT-AND-PASTE

TH. $\rightarrow$ ALG. GREEDY È OTTIMO

PROV. $\rightarrow f_1 \dots f_n$, ordinati $d_1 \leq d_2 \leq \dots \leq d_n$
 $\sum_{i=1}^k d_i \leq c$ | EXCHANGE ARGUMENT

$\rightarrow$ SUPPONIAMO PER ASSURDO $\sum_{i=1}^k d_i > c$

$O = \{f_1 \dots f_n\} \rightarrow \underline{m > k}$

OPT $\Rightarrow 1 \dots m < k$ FUORI DAL RANGO $\{1 \dots n\}$

$f_1 \dots f_m \Rightarrow m > k \rightarrow \text{NO OPT}$

$O \Rightarrow S^* \setminus \{f_i\}$

① TUTTI I PLUS SONO IN O

$$\sum_{i=1}^{k-1} d_i > c \quad (m-k) \text{ PLUS IN PIÙ}$$

② $\exists f_i \in \{f_1, \dots, f_m\} \mid f_i \notin O$

$$\underline{O' = O \setminus \underbrace{\{f_i\}}_{\text{NON BUONA}} \cup \{f_i\}}_{\text{BUONA}} \leq c$$

Domanda A (7 punti) Dare la definizione formale della classe $\Omega(f(n))$ per una funzione $f(n)$. Mostrare che date due funzioni $f(n), g(n)$ (che si possono assumere sempre positive), se $f(n) = \Omega(n^2)$ allora $f(n) + g(n) = \Omega(n^2 + g(n))$. Vale anche $f(n) - g(n) = \Omega(n^2 - g(n))$? Dimostrarlo o fornire un controesempio.

$$- f(n) = \Omega(n^2) \Rightarrow f(n) + g(n) = \Omega(n^2 + g(n))$$

sol. $\Omega(f(n)) \rightarrow g(n) \leq c$

$$\exists c > 0, \exists n_0 \in \mathbb{N} : \forall n \geq n_0$$

$$0 \leq c \cdot f(n) \leq g(n)$$

$$\Omega(n^2) \rightarrow 0 \leq c \cdot f(n^2) \leq g(n^2)$$

$$\Omega[n^2 + g(n)] \rightarrow \left[0 \leq c \cdot f(n^2) + g(n) \right] \leq g(n^2) + \underbrace{g(n)}_{>0}$$

PROOF $\rightarrow \underbrace{f(n) + g(n)}_{\text{SOERA}} = \Omega(n^2 + g(n))$

$$\exists c_1 = \min \{c, 1\}$$

$$f(n^2) + g(n^2) \geq c_1 \cdot (n^2 + g(n))$$

$$\begin{aligned} & \text{~~~~~} \geq 0 \qquad \geq C \cdot n^2 + \underbrace{C \cdot g(n)}_{>0} \\ & \qquad \qquad \qquad \forall n > 0, \forall C > 1 \end{aligned}$$

Domanda A (7 punti) Dare la definizione formale della classe $\Omega(f(n))$ per una funzione $f(n)$. Mostrare che date due funzioni $f(n), g(n)$ (che si possono assumere sempre positive), se $f(n) = \Omega(n^2)$ allora $f(n) + g(n) = \Omega(n^2 + g(n))$. Vale anche $f(n) - g(n) = \Omega(n^2 - g(n))$? Dimostrarlo o fornire un controesempio.

$$f(n) - g(n) = \Omega(n^2 - g(n))$$

$$0 \leq C \cdot (n^2 - \underbrace{g(n)}_{>0}) \leq f(n) - g(n)$$

$$C \cdot n^2 \leq \underbrace{f(n)}_{>0} - \underbrace{g(n)}_{>0}$$

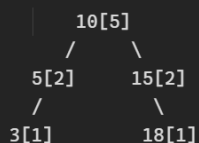
DIPENDE DA C...
f, g POSITIVE
(MA NON ORDINATE!)

Dato un albero binario di ricerca T, si vuole arricchirlo con un campo `x.descendants` che, per ogni nodo x, memorizza il numero di nodi nel sottoalbero radicato in x (incluso x stesso).

(a) Scrivere una funzione `UpdateDescendants(T, x)` che, dato un nodo x appena inserito (foglia), aggiorna il campo `descendants` di x e di tutti i suoi antenati. Valutare la complessità.

(b) Scrivere una funzione `RankOf(T, k)` che, dato l'albero T arricchito e una chiave k presente in T, restituisce la posizione (rank) di k nell'ordinamento delle chiavi dell'albero, dove il rank del minimo è 1. Valutare la complessità.

Esempio:



`RankOf(T, 15) = 4` perché 15 è il 4° elemento nell'ordinamento {3, 5, 10, 15, 18}

- `UpdateDescendants(T, x)`

// `x.left, x.right ... x.desc`

`x.desc = 1`
`y = x.p`

while `y != nil`
 `y.desc += 1`
 `y = y.p`

- `RankOf(T, k)`

`rank = 0`

while `x.key != k.key`
 if `k.key < x.key`
 `x = x.left`
 else
 `x = x.right`
 `rank ++`

return `rank`

algorithm `BruteForceRank(node, x)`:

// INPUT
// `x` = the value whose rank we want to determine
// `node` = the root node of the (sub-)tree
// for which we're calculating the rank of `x`
// OUTPUT
// `rank` = the rank of `x` in the (sub-)tree whose root is `node`

if `node = NONE`:
 return 0

// first, count the nodes that are $\leq x$ in the left sub-tree
`rank_left` $\leftarrow$ `BruteForceRank(node.left, x)`

// then, count the nodes that are $\leq x$ in the right sub-tree
`rank_right` $\leftarrow$ `BruteForceRank(node.right, x)`

if `node.value` $\leq x$:
 return 1 + `rank_left` + `rank_right`
else:
 return `rank_left` + `rank_right`

<https://www.baeldung.com/cs/node-rank-bst>

algorithm `OrderBasedCalculationOfRank(node, x)`:

// INPUT
// `x` = the value whose rank we want to determine
// `node` = the root node of the (sub-)tree
// for which we're calculating the rank of `x`
// OUTPUT
// `rank` = the rank of `x` in the (sub-)tree whose root is `node`

if `node = NONE`:
 return 0

if `node.value` $\leq x$:
 return 1 + `Rank(node.left, x)` + `Rank(node.right, x)`
else:
 return `Rank(node.left, x)`

Data una stringa $X = (x_1, x_2, \dots, x_n)$, si consideri la seguente quantità $p(i, j)$ definita per $1 \leq i \leq j \leq n$:

$$p(i, j) = \begin{cases} 0 & \text{se } i = j \\ 0 & \text{se } i+1 = j \wedge x_i = x_j \\ 1 + \min(p(i+1, j), p(i, j-1)) & \text{altrimenti} \end{cases}$$

La quantità $p(1, n)$ rappresenta il numero minimo di caratteri da aggiungere alla stringa X per renderla palindroma.

- (a) Scrivere una coppia di algoritmi `INIT_P(n)` e `REC_P(X, i, j)` per il calcolo memoizzato di $p(1, n)$.
- (b) Scrivere un algoritmo iterativo **bottom-up** `BOTTOM_UP_P(X)` che calcola $p(1, n)$.
- (c) Valutare la complessità temporale e spaziale dell'algoritmo del punto (b).

```

INIT_P(n)
    crea matrice P[1..n][1..n]
    if n = 0 return 0
    for i = 1 to n
        P[i, i] = 0
        P[i, i+1] = 0
    return P

REC_P(X, i, j, P)
    if P[i, j] = 0
        if X[i] = X[j]
            P[i][j] =
                REC_P(X, i+1, j-1, P)
        else
            P[i][j] =
                1 + min(REC_P(X, i+1, j, P),
                    REC_P(X, i, j-1, P))
    return P[i, j]

```

`BOTTOM_UP_P(X)`

```

1. n = length(X)
2. crea matrice P[1..n][1..n]
3. // Casi base: sottostr lunghe 1
4. for i = 1 to n
5.     P[i][i] = 0
6. // Sottostr lunghe 2
7. for i = 1 to n-1
8.     if X[i] = X[i+1]
9.         P[i][i+1] = 0
10.    else
11.        P[i][i+1] = 1
12. // Sottostr lunghe 3..n
13. for i = 2 to n
14.     for j = i+1 to n
15.         if X[i] = X[j]
16.             P[i][j] = P[i+1][j-1]
17.         else
18.             P[i][j] = 1 + min(P[i+1][j], P[i][j-1])
19. return P[1][n]

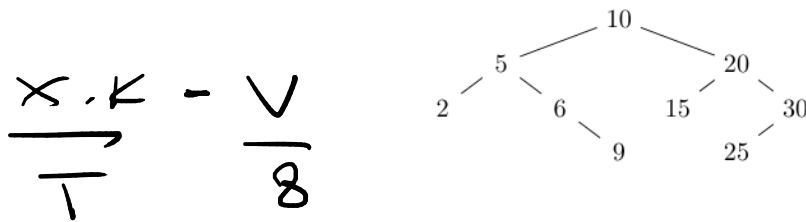
```

$$p(i, j) = \begin{cases} 0 & \text{se } i = j \\ 0 & \text{se } i+1 = j \wedge x_i = x_j \\ 1 + \min(p(i+1, j), p(i, j-1)) & \text{altrimenti} \end{cases}$$

SPC

$$O(n^2) = \begin{matrix} n/2 & n/2 \\ \swarrow & \searrow \\ L & J & L & J \end{matrix}$$

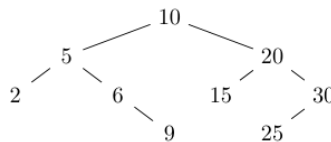
Esercizio 1 (10 punti) Realizzare una funzione `mdist(T,v)` che dato un albero binario di ricerca `T` e una chiave `v` restituisce un nodo `x` di `T` la cui chiave abbia distanza minima da `v`, ovvero tale che il valore $|x.k - v|$ sia minimo. Fornire lo pseudocodice, valutarne la complessità e giustificarne la correttezza. Ad esempio se `T` è l'albero sotto indicato, allora `mdist(T,8)` ritorna il nodo con chiave 9 mentre `mdist(T,28)` ritorna il nodo con chiave 30.



$$|10 - 8| = 2 \quad |20 - 8| = 12$$

min w.r.t
2

Esercizio 1 (10 punti) Realizzare una funzione `mdist(T,v)` che dato un albero binario di ricerca `T` e una chiave `v` restituisce un nodo `x` di `T` la cui chiave abbia distanza minima da `v`, ovvero tale che il valore $|x.k - v|$ sia minimo. Fornire lo pseudocodice, valutarne la complessità e giustificarne la correttezza. Ad esempio se `T` è l'albero sotto indicato, allora `mdist(T,8)` ritorna il nodo con chiave 9 mentre `mdist(T,28)` ritorna il nodo con chiave 30.



```
mDist(T, v)
  x = T.root
  return mDist_aux(x, v, best)
```

```
mDist_aux(x, v, best)
  if x = nil return nil
```

```
// Calcolo distanza corrente
curr_dist = |x.key - v|
min_dist = 0
```

```
// Aggiorno nodo se necessario
if curr_dist < min_dist
  min_dist = curr_dist
  best = x
```

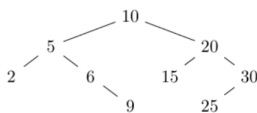
```
// Se dist = 0 → trovato best!
if curr_dist == 0 return best
```

```
// Decidi sottoalbero dove muoverci
```

```
if v < x.key
  best = m_dist_aux(x.left, v, best)
  //Se dx, lì sol. migliore
  if x.right && (x.key - v < min_dist)
    best = m_dist_aux(x.right, v, best)
```

```
else
  best = m_dist_aux(x.right, v, best)
  if x.left && (x.key - v < min_dist)
    best = m_dist_aux(x.left, v, best)
```

Esercizio 1 (10 punti) Realizzare una funzione `mdist(T,v)` che dato un albero binario di ricerca `T` e una chiave `v` restituisce un nodo `x` di `T` la cui chiave abbia distanza minima da `v`, ovvero tale che il valore $|x.k - v|$ sia minimo. Fornire lo pseudocodice, valutarne la complessità e giustificarne la correttezza. Ad esempio se `T` è l'albero sotto indicato, allora `mdist(T,8)` ritorna il nodo con chiave 9 mentre `mdist(T,28)` ritorna il nodo con chiave 30.



Date due stringhe $X = (x_1, \dots, x_m)$ e $Y = (y_1, \dots, y_n)$, una **longest common substring** è una sottostringa **consecutiva** di lunghezza massima che appare in entrambe.

Definire la quantità $L(i, j)$ per $1 \leq i \leq m$, $1 \leq j \leq n$ come:

$$L(i, j) = \begin{cases} 0 & \text{se } i = 0 \vee j = 0 \\ L(i-1, j-1) + 1 & \text{se } x_i = y_j \\ 0 & \text{se } x_i \neq y_j \end{cases}$$

La lunghezza della LCS substring è: $\max\{L(i, j) : 1 \leq i \leq m, 1 \leq j \leq n\}$

(a) Qual è la complessità dell'algoritmo esaustivo che analizza tutte le possibili sottostringhe comuni?

(b) Assumendo di conoscere un algoritmo che determina se una stringa di m caratteri è sottostringa di un'altra stringa di n caratteri in tempo $O(m+n)$, come si può modificare l'algoritmo per renderlo più efficiente?

(c) Scrivere un algoritmo di programmazione dinamica **bottom-up** più efficiente di quello del punto precedente. Sono richiesti: relazione di ricorrenza sulle lunghezze (senza dimostrazione) e algoritmo bottom-up. Valutare la complessità.

Esempio:

$X = \text{"ABCXYZ"}$

$Y = \text{"XYZABC"}$

$\text{LCS substring} = \text{"ABC"}$ (lunghezza 3)

Domanda B (7 punti) Si consideri una tabella hash di dimensione $m = 7$, e indirizzamento aperto con doppio hash basato sulle funzioni $h_1(k) = k \bmod m$ e $h_2(k) = 1 + k \bmod (m - 2)$. Si descriva sinteticamente come avviene l'inserimento degli elementi e si specifichi il risultato dell'inserzione della sequenza di chiavi: 10, 20, 34, 35, 48.

Sarebbe appropriato lavorare con una tabella di dimensione $m = 8$ e le stesse funzioni hash?

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

Indice	Chiave	Probing usato
0	35	i=0
1	48	i=4
2	-	-
3	10	i=0
4	34	i=1
5	-	-
6	20	i=0

$$h_1(k) = k \bmod 7 \text{ DISPARI}$$

$$h_2(k) = 1 + k \bmod (7 - 2)$$

$\sum$
DISP.

3 / 5 $\rightarrow$ COLUS.

$$\left[\begin{array}{l} (k \bmod 2) \\ 1 + (k \bmod 6) \end{array} \right] \rightarrow 10, 20, 34, 35, 48$$

PROBLEMA $\rightarrow$ PROBING = SCANSIONE

Supponiamo k tale che $h_1(k) = 0$ e $h_2(k) = 4$

Sequenza di probing con $m = 8$:

- $i=0$: $(0 + 0 \cdot 4) \bmod 8 = 0$
- $i=1$: $(0 + 1 \cdot 4) \bmod 8 = 4$
- $i=2$: $(0 + 2 \cdot 4) \bmod 8 = 0 \leftarrow$ ciclo!
- $i=3$: $(0 + 3 \cdot 4) \bmod 8 = 4 \leftarrow$ ciclo!

Visita solo $\{0, 4\} \rightarrow$ non copre tutta la tabella!

SOLO PARI

SOLO
DISPARI

NO
VISITA POSIZIONI

$m = 7 \rightarrow$ PRIMO

(3, 5) $\Rightarrow$

(COPIA)

$\rightarrow$ PARI
TRAD
LORO

****Regola generale:****

Per doppio hashing, scegliere ****m primo****

oppure garantire che $h_2(k)$ sia sempre coprimo con m .

Dati due max-heap $H_1[1..n]$ e $H_2[1..n]$ della stessa dimensione, costruire un nuovo max-heap $H[1..2n]$ contenente gli elementi alternati: prima il massimo di H_1 , poi il massimo di H_2 , poi il secondo massimo di H_1 , ecc.

Implementare `AlternateMerge(H1, H2)` e valutarne la complessità.

Esercizio 1 (10 punti) Siano dati due array $A[1..2n]$ e $B[1..n]$ organizzati a max-heap, entrambi contenenti n elementi ($\text{heapsize}=n$). Realizzare una procedura $\text{SortJoin}(A, B, n)$ che dati in input array A e B con le proprietà sopra descritte, ritorna in A un array ordinato contenente tutti i $2n$ elementi originariamente presenti in A e B . L'array B può essere modificato durante l'esecuzione della procedura, se necessario, ma l'algoritmo dovrà operare in *spazio costante*. Dare lo pseudocodice della procedura, motivarne la correttezza e valutarne la complessità. Se si utilizzano operazioni sui max-heap andranno definite esplicitamente.

Esercizi

Esercizio 1 (9 punti) In questo esercizio si considerano alberi binari di ricerca (BST) completi, memorizzati in array con la stessa convenzione che si usa normalmente per la memorizzazione di uno heap in un array.

Realizzare una funzione $\text{max}(T, n)$ che determina il massimo di un array T di dimensione n che memorizza un BST completo.

Scrivere quindi una funzione $\text{merge}(T1, T2, k)$ che dati due array $T1$ e $T2$ di dimensione n , che memorizzano BST completi e una chiave k maggiore di tutte le chiavi di $T1$ e minore di tutte quelle di $T2$, restituisce un array (di dimensione $2n+1$) che memorizza un BST che contiene tutte le chiavi di $T1$ e $T2$, e la chiave k .

In entrambi i casi motivare la correttezza delle funzioni e valutarne la complessità.